

智能垃圾分类处理系统仿真程序设计

成绩

指导教师：罗志祥

专业班级：光电信息科学与工程 202409

姓名：陈恪瑾

学号：U202413925

联系电话：18271851291

1 任务要求

1.1 实验目的

- 掌握使用 AT89S52 的 I/O 口；
- 理解 AT89S52 单片机的基本结构和工作原理；
- 掌握分支结构，条件转移程序设计方法；
- 具备调试程序的基本技能。

1.2 实验内容

在 Keil 环境中设计一个基于 AT89S52 单片机的智能垃圾分类处理系统，用户通过 P2 口的输入和 P1 口输出（上电复位初始状态为全 1），模拟小区垃圾回收终端系统的运行状态，系统需要实现以下功能：

- 小区居民通过键入相应按键选择垃圾类型（分别用 P2.4~P2.7 端口模拟按键输入，低电平有效，P2.7=0、P2.6=0、P2.5=0、P2.4=0 分别代表“塑料纸张”、“玻璃金属”、“厨余垃圾”和“危害垃圾”四类）。
- P1.0~P1.3 四位代表垃圾桶盖开关控制，控制名为“塑料纸张”、“玻璃金属”、“厨余垃圾”和“危化品垃圾”四个垃圾桶盖，高电平为关闭状态，低电平为打开状态。
- 垃圾桶盖每次打开的时间缺省值为 8s，然后自动关闭，请采用循环程序配合 NOP 指令完成延时子程序，精度控制在 0.1s 以内。

请编写程序，实现用户键入，控制相应的垃圾桶盖打开和关闭。

1.3 提高部分（申优选做）

- 系统初始化后，随机生成 4 个 0~9 的整数，置于寄存器 R0~R3 中表示四个垃圾桶的剩余容量。用户每次投递之后，容量相应减少。当投放垃圾超出剩余容量后，对

应的标志位发生变化（自行定义），便于提醒物业及时清理，此时再次按键投放时，该桶盖不再打开避免溢出。

- 其他个性化设计。

2 设计思路

本系统基于 AT89S52 单片机核心架构，采用模块化程序设计思想，围绕垃圾分类投放的核心需求，拆分功能模块实现全流程控制，整体设计逻辑如下：

1. **系统初始化模块：**上电后配置 P1 口为高电平 0xFF（所有桶盖默认关闭），并清零位于 20H 内存上的 4 个垃圾桶满载标志位，将其同步至 P3 口状态以控制指示灯。设置定时器 T0 工作在模式 1 并启动，将其自由运行的计数值作为随机数生成的扰动源，随后调用伪随机数生成子程序，为 4 个垃圾桶生成 0-9 的初始剩余容量（储存在 R0~R3 寄存器中）。
2. **按键检测与分支跳转模块：**在主循环中，直接使用 JNB 指令轮询扫描 P2.7~P2.4 口的低电平信号（输入有效）。若检测到对应按键被按下，主循环将直接跳转至相应类型垃圾的投放处理分支，否则将持续循环扫描按键。
3. **垃圾投放与容量管理模块：**在具体的投放处理分支中，系统首先检查 20H 寄存器中对应的满载标志。若标志位已置 1 或当前容量寄存器减量前已为 0，表示垃圾桶已满，系统拒绝开盖，进入按键释放等待逻辑；若未满，将对应容量减 1（减至 0 时同步更新满载标志并点亮 LED），拉低 P1 口对应引脚即可开启桶盖。此时系统调用延时子程序保持 8 秒，随后拉高 P1 口关闭桶盖，并强制等待按键释放避免按键连发。
4. **多重循环延时子程序：**12MHz 标称晶振下，利用嵌套循环辅以 NOP 空操作来消耗指令周期：底层为一个百毫秒级的延时函数（内部进行 166×200 次的 DJNZ 定周期减量跳转，估算时长 100ms），外围固定调用该子程序 80 次产生误差接受范围内的 8 秒延迟，拼凑出盖板打开维持时间。
5. **硬件定时随机与指示模块：**读取自由不受控的定时器内置 TL0、TH0 计数器的值充当随机数引子，经过多项常数 XRL 异或扰动、RL/RR 循环移位与掩码操作后，采用循环减 10 的方式进行模 10 取余，得出 0-9 宽度的随机容量初值。同时采用掩码方式融合 20H 满载标志位写出至 P3 外设寄存器，实现在 P3.0 P3.3 端口满载时的 LED 发光指示预警。

3 资源分配

3.1 I/O 口硬件资源分配

I/O 引脚	方向	功能定义	有效电平
P1.0	输出	塑料纸张垃圾桶盖控制	低电平打开，高电平关闭
P1.1	输出	玻璃金属垃圾桶盖控制	低电平打开，高电平关闭
P1.2	输出	厨余垃圾垃圾桶盖控制	低电平打开，高电平关闭
P1.3	输出	危害垃圾垃圾桶盖控制	低电平打开，高电平关闭
P2.4	输入	危害垃圾投放按键	低电平有效
P2.5	输入	厨余垃圾投放按键	低电平有效
P2.6	输入	玻璃金属投放按键	低电平有效
P2.7	输入	塑料纸张投放按键	低电平有效
P3.0	输出	塑料纸张满载指示信号	高电平点亮
P3.1	输出	玻璃金属满载指示信号	高电平点亮
P3.2	输出	厨余垃圾满载指示信号	高电平点亮
P3.3	输出	危害垃圾满载指示信号	高电平点亮

3.2 寄存器资源分配

寄存器	功能定义
R0	存储塑料纸张垃圾桶剩余容量，初始值为 0-9 随机数
R1	存储玻璃金属垃圾桶剩余容量，初始值为 0-9 随机数
R2	存储厨余垃圾垃圾桶剩余容量，初始值为 0-9 随机数
R3	存储危害垃圾垃圾桶剩余容量，初始值为 0-9 随机数
R5~R7	延时子程序的循环计数寄存器（R5/R6 定百毫秒，R7 定 8 秒时长）
TH0, TL0	定时器 0 的计数器，自由运行以提供生成随机数的硬件扰动源
累加器 A	临时存储 I/O 口状态、参与位运算防误触、数据中转及随机数运算暂存

3.3 片内 RAM 资源分配

存储单元	功能定义
20H.0	塑料纸张垃圾桶满载标志位，置 1 表示容量用尽
20H.1	玻璃金属垃圾桶满载标志位，置 1 表示容量用尽
20H.2	厨余垃圾垃圾桶满载标志位，置 1 表示容量用尽
20H.3	危化品垃圾垃圾桶满载标志位，置 1 表示容量用尽
21H 单元	刷新 P3 口满载 LED 指示状态时，用作掩码和数据传递的临时缓存

4 流程图

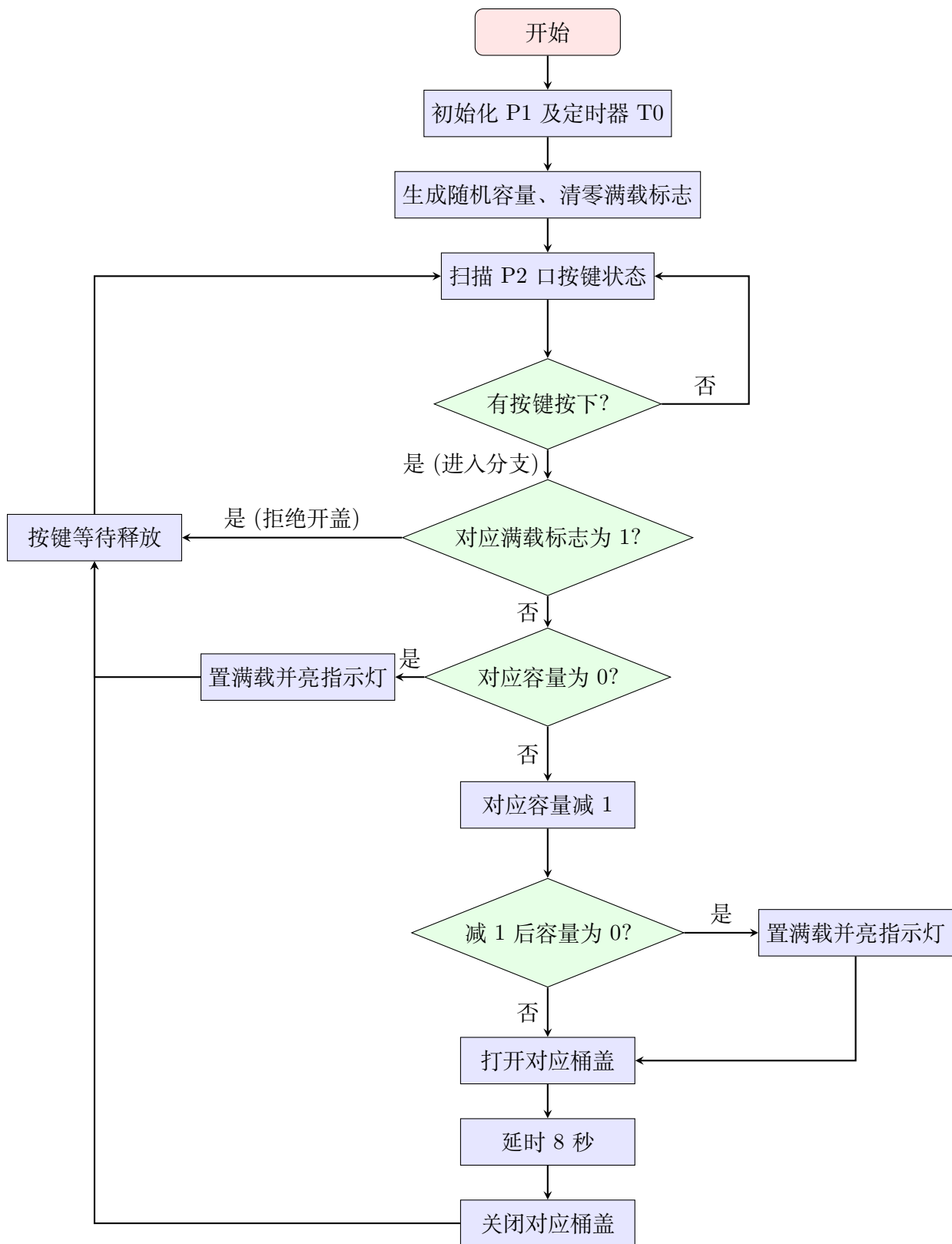


图 1: 主循环及投放处理分支流程图

5 源代码（含文件头说明、语句行注释）

- ； 小区居民通过键入相应按键选择垃圾类型（分别用P2.4~P2.7端口模拟按键输入，低电平有效，P2.7=0、P2.6=0、P2.5=0、P2.4=0分别代表“塑料纸张”、“玻璃金属”、“厨余垃圾”和“危害垃圾”四类）。
- ； P1.0~P1.3四位代表垃圾桶盖开关控制，控制名为“塑料纸张”、“玻璃金属”、“厨余垃圾”和“危化品垃圾”四个垃圾桶盖，高电平为关闭状态，低电平为打开状态。
- ； P3.0~P3.3: 4 个满载指示灯输出
- ； R0~R3: 4 个桶当前剩余容量（范围 0~9）
- ； 20H bit0~bit3: 4 个桶的满载标志位（1=满载）

```
ORG      0000H
```

```
LJMP     START
```

START:

； 上电默认关闭全部盖板（P1.0~P1.3 置 1）

```
MOV      P1, #0FFH
```

； 清空满载标志字节，并刷新到 P3 指示灯

```
MOV      20H, #00H
```

```
LCALL    UPDATE_FULL_LED
```

； 启动 T0 作为伪随机扰动源

； TMOD=01H -> 定时器0工作在模式1（16位）

```
MOV      TMOD, #01H
```

```
SETB     TR0
```

； 为 4 个桶生成初始容量（0~9）

```
LCALL    RAND_0_9
```

```
MOV      R0, A
```

```
LCALL    RAND_0_9
```

```
MOV      R1, A
```

```
LCALL    RAND_0_9
```

```
MOV      R2, A
```

```
LCALL    RAND_0_9
```

```

        MOV        R3, A

MAIN_LOOP:
        ; 按键扫描
        JNB        P2.7, KEY_BIN0
        JNB        P2.6, KEY_BIN1
        JNB        P2.5, KEY_BIN2
        JNB        P2.4, KEY_BIN3
        SJMP       MAIN_LOOP

KEY_BIN0:
        LJMP       TRY_BIN0

KEY_BIN1:
        LJMP       TRY_BIN1

KEY_BIN2:
        LJMP       TRY_BIN2

KEY_BIN3:
        LJMP       TRY_BIN3

TRY_BIN0:
        ; 若 bin0 已满 (bit0=1), 拒绝开盖
        MOV        A, 20H
        ANL        A, #01H
        JNZ        BINO_DENY

        ; 若容量已经是 0, 则直接标记满载
        MOV        A, R0
        JZ         BINO_MARK_FULL

        ; 成功投放一次, 容量减 1
        DEC        R0
        MOV        A, R0
        JNZ        BINO_OPEN

```

; 若减到 0, 置满载标志并刷新 LED

ORL 20H, #01H

LCALL UPDATE_FULL_LED

BINO_OPEN:

; 开盖 8 秒后关盖

ANL P1, #0FEH

LCALL DELAY_8S

ORL P1, #01H

LCALL KEY_AUTO_RESET

LCALL WAIT_RELEASE

LJMP MAIN_LOOP

BINO_MARK_FULL:

ORL 20H, #01H

LCALL UPDATE_FULL_LED

BINO_DENY:

; 拒绝时也等待按键释放, 避免连发

LCALL WAIT_RELEASE

LJMP MAIN_LOOP

TRY_BIN1:

; 若 bin1 已满 (bit1=1), 拒绝开盖

MOV A, 20H

ANL A, #02H

JNZ BIN1_DENY

; 若容量已经是 0, 则直接标记满载

MOV A, R1

JZ BIN1_MARK_FULL

; 成功投放一次, 容量减 1

DEC R1


```

MOV     A, R1
JNZ     BIN1_OPEN

; 若减到 0, 置满载标志并刷新 LED
ORL     20H, #02H
LCALL   UPDATE_FULL_LED

```

BIN1_OPEN:

```

; 开盖 8 秒后关盖
ANL     P1, #0FDH
LCALL   DELAY_8S
ORL     P1, #02H
LCALL   KEY_AUTO_RESET
LCALL   WAIT_RELEASE
LJMP    MAIN_LOOP

```

BIN1_MARK_FULL:

```

ORL     20H, #02H
LCALL   UPDATE_FULL_LED

```

BIN1_DENY:

```

LCALL   WAIT_RELEASE
LJMP    MAIN_LOOP

```

TRY_BIN2:

```

MOV     A, 20H
ANL     A, #04H
JNZ     BIN2_DENY

MOV     A, R2
JZ      BIN2_MARK_FULL

DEC     R2
MOV     A, R2

```

```
JNZ      BIN2_OPEN

        ORL      20H, #04H
        LCALL    UPDATE_FULL_LED

BIN2_OPEN:
        ANL      P1, #0FBH
        LCALL    DELAY_8S
        ORL      P1, #04H
        LCALL    KEY_AUTO_RESET
        LCALL    WAIT_RELEASE
        LJMP     MAIN_LOOP

BIN2_MARK_FULL:
        ORL      20H, #04H
        LCALL    UPDATE_FULL_LED

BIN2_DENY:
        LCALL    WAIT_RELEASE
        LJMP     MAIN_LOOP

TRY_BIN3:
        MOV      A, 20H
        ANL      A, #08H
        JNZ      BIN3_DENY

        MOV      A, R3
        JZ       BIN3_MARK_FULL

        DEC      R3
        MOV      A, R3
        JNZ      BIN3_OPEN

        ORL      20H, #08H
```

```
LCALL    UPDATE_FULL_LED
```

```
BIN3_OPEN:
```

```
ANL      P1, #0F7H
LCALL    DELAY_8S
ORL      P1, #08H
LCALL    KEY_AUTO_RESET
LCALL    WAIT_RELEASE
LJMP     MAIN_LOOP
```

```
BIN3_MARK_FULL:
```

```
ORL      20H, #08H
LCALL    UPDATE_FULL_LED
```

```
BIN3_DENY:
```

```
LCALL    WAIT_RELEASE
LJMP     MAIN_LOOP
```

; 伪随机子程序: 返回 $A=0\sim9$

; 思路: 使用 TLO/THO 做扰动, 做若干异或与移位后, 采用重复减 10 得到模 10

```
RAND_0_9:
```

```
MOV      A, TLO
XRL      A, THO
XRL      A, #05AH
RL       A
XRL      A, #01DH
RR       A
ANL      A, #01FH
```

```
RAND_MOD10:
```

```
CLR      C
SUBB     A, #0AH
JNC      RAND_MOD10
ADD      A, #0AH
```

RET

; 将 20H 的 bit0~bit3 同步到 P3.0~P3.3 (P3 高四位保持原值)

UPDATE_FULL_LED:

MOV A, 20H

ANL A, #0FH

MOV 21H, A

MOV A, P3

ANL A, #0F0H

ORL A, 21H

MOV P3, A

RET

; 8 秒延时: 80 次 100ms

DELAY_8S:

MOV R7, #80

DELAY_8S_LOOP:

LCALL DELAY_100MS

DJNZ R7, DELAY_8S_LOOP

RET

; 粗略 100ms 延时 (按 12MHz 经典 8051 估算)

DELAY_100MS:

MOV R5, #166

D100_OUTER:

MOV R6, #200

D100_INNER:

NOP

DJNZ R6, D100_INNER

DJNZ R5, D100_OUTER

RET

; 等待 P2.7~P2.4 全部释放 (都为 1)

WAIT_RELEASE:

```
MOV    A, P2

ANL    A, #0FOH

CJNE   A, #0FOH, WAIT_RELEASE

RET
```

；关盖后自动将按键输入位复位为高电平状态（释放 P2.7~P2.4）

KEY_AUTO_RESET:

```
ORL    P2, #0FOH

RET

END
```

6 程序测试方法与结果

采用 keil 软件进行仿真测试，效果如下：

程序启动后，根据随机数生成算法，R0 R3 寄存器中分别存储了 0-9 的随机整数，表示四个垃圾桶的剩余容量。

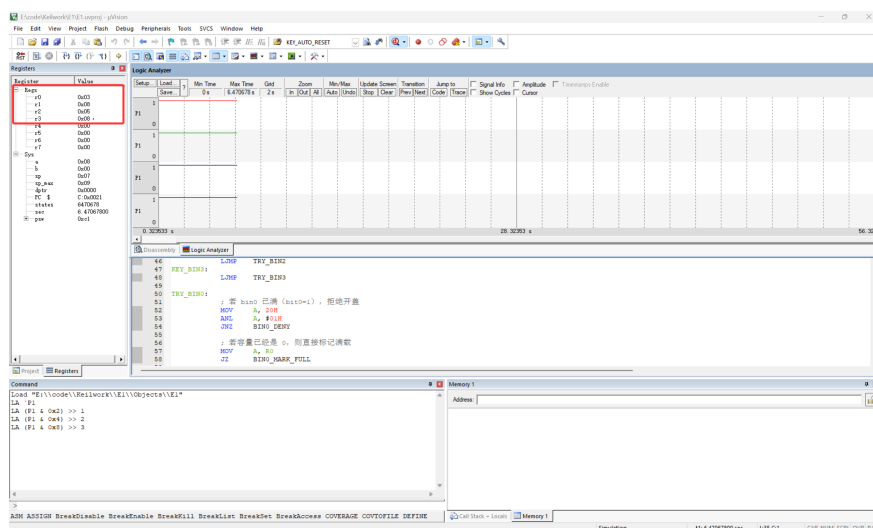


图 2: 初始随机垃圾桶容量分布

按下 P2.7 对应的按键，将 P2.7 调为低电平时，观察到 P1.0 也变为低电平，表示打开塑料纸张垃圾桶。

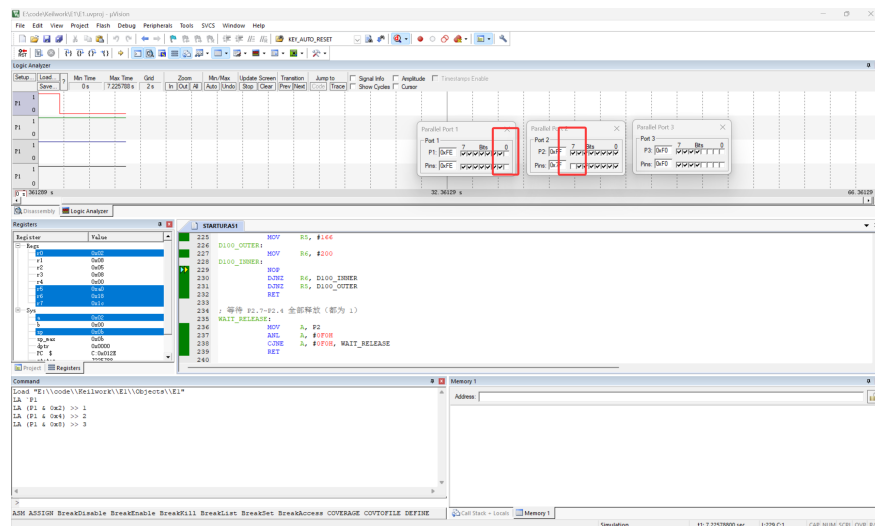


图 3: 按下 P2.7, P1.0 变为低电平

根据波形图查看延时效果, 可以看到 P1.0 在被拉低后保持了约 8 秒 ($10.02458s - 2.0176173s = 8.0069627s$, 符合误差 1s 以内的要求) 的时间, 然后自动拉高关闭。

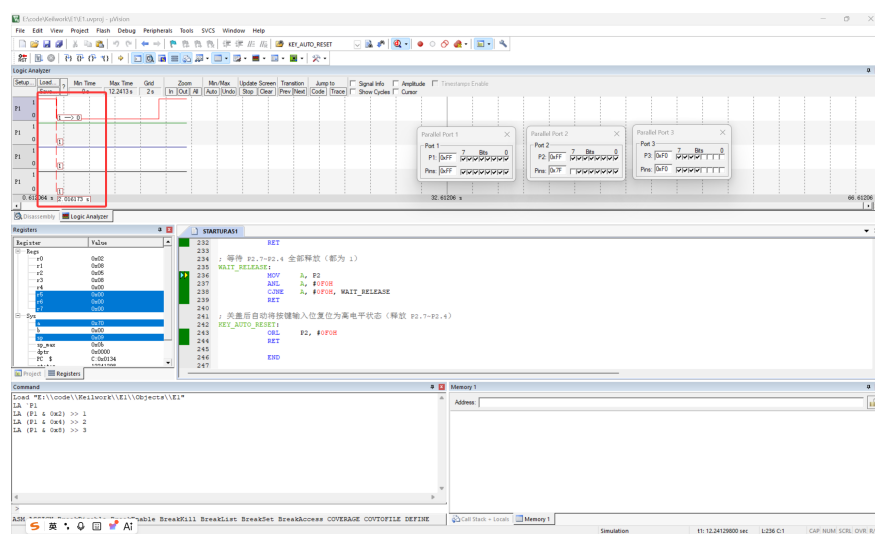


图 4: 电平下降沿

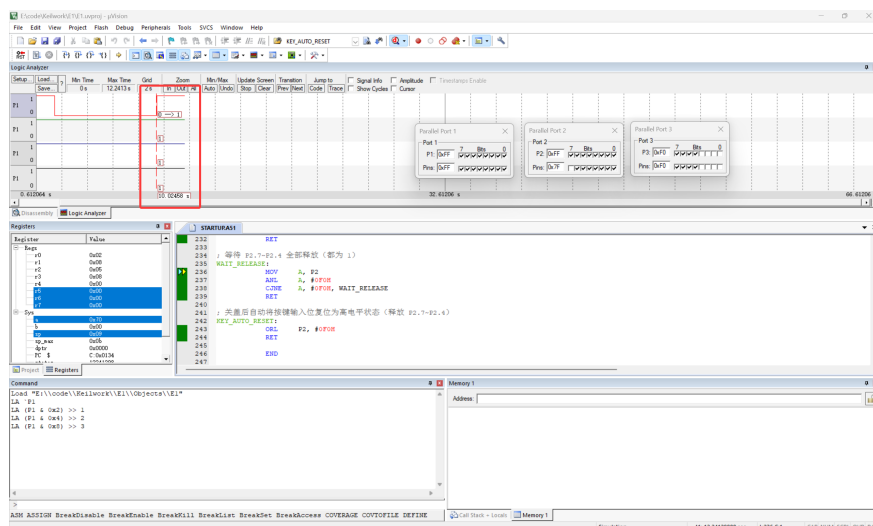


图 5: 电平上升沿

将塑料纸张对应的垃圾桶容量耗尽，即当 R0 为 0 时，此时调节 P2.7=0，P1.0 不响应按键，并且将 P3.0 电平拉高提醒垃圾清理（可以在此引脚外接 LED 或有源蜂鸣器发出声光信号提示）。

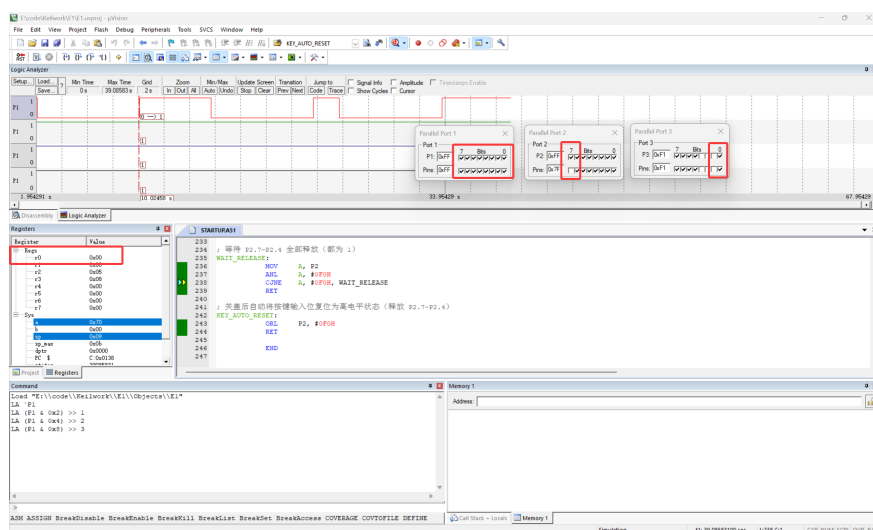


图 6: 垃圾桶满载指示

7 实验问题分析与对策

1. 伪随机数生成的随机性问题：

- **问题：**在单片机系统中生成真正的随机数较为困难，常规算术方法容易在每次复位后产生固定的初始序列，导致每次开机容量一致。

- **对策：**在课上受到启发并查阅资料后得知，可以利用系统不可预知的物理运行状态作为扰动源产生随机数。具体来说，程序初始化后开启定时器 T0 自由运行，在提取随机数时，读取处于自由变换中的 TH0 和 TL0 计数值。但是这样产生的随机数较容易预测，此时通过常数异或、循环移位等位运算打乱数据规律，最后通过循环减 10 求模，成功获得了在 0 9 之间分布且难以预测的随机初始容量。但是这样产生的随机数仍然是伪随机，在实际应用中如果希望获得真随机数可以使用外部随机数发生模块获得真随机数。

2. SJMP 跳转问题：

- **问题：**在程序设计过程中，初始版本在主程序扫描到对应按键后，直接使用 SJMP 指令跳转到对应的投放处理分支。由于 SJMP 只能跳转-128 到 +127 字节范围内的地址，导致部分分支因距离过远无法正确跳转，按下按键后垃圾桶盖始终无法被打开。
- **对策：**将 SJMP 指令替换为 LJMP 指令，允许跳转到任意地址范围内的分支处理程序，成功解决了跳转异常问题，保证了程序的正确执行流程。

3. 按键自动复位问题：

- **问题：**程序设计时，我希望设计按键按下后会自动复位为高电平，于是设计了程序

```
KEY_AUTO_RESET:
    ORL    P2, #0F0H
    RET
```

希望通过将 P2 口的低 4 位强制置 1 来实现按键自动复位，但在实际测试中发现，这段代码只能复位 P2 对应的寄存器，并不能真正改变外部硬件引脚的电平状态。

- **对策：**经过仔细思考，我意识到按键的电平状态是由外部硬件电路决定的，单片机内部的指令无法直接改变外部引脚的电平。在仿真中无法模拟出按键自动复位的效果，因此在仿真过程中需要手动将按键对应的 P2 口引脚拉高来模拟按键释放的状态。

本人承诺: 本报告内容真实, 无伪造数据, 无抄袭他人成果。本人完全了解学校相关规定, 如若违反, 愿意承担其后果。

签字:

陈恪瑾

2026 年 4 月 26 日